Tesserae Ventures
SECURITY RESEARCH

Honest Validator Self-Equivocation in TON Simplex Consensus

SkipVote + NotarizeVote broadcast for the same slot via two independent code paths, violating the single-vote invariant without Byzantine behaviour.

Repository	Branch	Commit	Scope	Date
ton-blockchain/ton	testnet	3bb6abc	validator/consensus/simplex/	14 March 2026

⚠ SEVERITY: HIGH — SAFETY INVARIANT VIOLATION, NO BYZANTINE ACTOR REQUIRED

Executive Summary

An honest Simplex validator can broadcast both a `SkipVote` and a `NotarizeVote` for the same consensus slot, producing a self-equivocation without any compromise or Byzantine behaviour. This violates the core Simplex safety invariant: *a validator must vote at most once per slot*.

The bug manifests through two independent code paths sharing the same root cause — an incomplete boolean guard that checks `!voted_final` but omits `!voted_notar` and `!voted_skip`:

- **Path A — `alarm()` race:** The alarm timer and `try_notarize()` coroutine use disjoint guards, allowing both to broadcast votes for the same slot concurrently.
- **Path B — `start_up()` restart:** On every restart, a validator broadcasts `SkipVote` for its active window slots without checking whether it already cast `NotarizeVote` before the crash.

Both paths are **dynamically confirmed** via the `test-consensus` harness.

Impact Assessment

PROPERTY	ASSESSMENT
Safety violation	Yes — self-equivocation from an honest validator; breaks Simplex's single-vote-per-slot invariant
Liveness impact	Yes — equivocating votes can poison slot pools; all N honest validators produce conflicting evidence simultaneously
Trigger (Path A)	Validation time exceeds alarm interval — no Byzantine actor required; occurs naturally under load
Trigger (Path B)	Any leader-window validator restart — fires 100% of the time the gremlin kills the leader (19/20 in test)
Affected scope	`validator/consensus/simplex/` — primary contest scope

Path A — `alarm()` Race Condition

Root Cause

`alarm()` and `try_notarize()` share state on a slot but use disjoint boolean guards. `alarm()` checks only `!voted_final` before broadcasting `SkipVote`. `try_notarize()` sets `voted_notar` and broadcasts `NotarizeVote` after resuming from `co_await ValidationRequest` without checking whether a `SkipVote` was sent during the suspension.

📄 `validator/consensus/simplex/consensus.cpp`

Bug A1 — `alarm()` : missing `!voted_notar` guard

```
// consensus.cpp ~line 144
td::actor::Task<> alarm() {
  for (size_t i = first_...; i < last_...; ++i) {
    auto slot = bus_->slots.get(i);
    if (slot && !slot->state->voted_final) {       // ← MUST also check !voted_notar && !voted_skip
      owning_bus().publish<BroadcastVote>(SkipVote{i});
    }
  }
}
```

Bug A2 — `try_notarize()` : missing `!voted_skip` check after co_await

```
// consensus.cpp ~line 228
td::actor::Task<> try_notarize(size_t slot_idx, ...) {
  auto res = co_await ValidationRequest{candidate};
  // ← alarm() may have fired during co_await and broadcast SkipVote
  slot->state->voted_notar = candidate->id;
  owning_bus().publish<BroadcastVote>(NotarizeVote{...});  // ← MUST check !voted_skip first
}
```

Bug A3 — `check_invariants()` : missing detection case

```
// pool.cpp ~line 209
void check_invariants() {
  if (finalize_.has_value() && skip_.has_value()) { ... report ... }
  if (finalize_.has_value() && notarize_.has_value()) { ... report ... }
  // MISSING: notarize_ + skip_ combination not detected:
  // if (notarize_.has_value() && skip_.has_value()) { report misbehavior; }
}
```

✓ DYNAMICALLY CONFIRMED

`pool.cpp:525 WARNING` — *"Dropping NotarizeVote{id={62, WQS7+...}} from validator 6 ... which references a finalized slot"*

Observed **1–500+ events per 30-second run**. All 8 validators affected across multiple runs. The slot finalised via `SkipCert` (produced from the equivocating validator's own `SkipVote`) before the `NotarizeVote` arrived — proving both vote types were broadcast for the same slot.

Path B — `start_up()` Restart Equivocation

Root Cause

On restart, `start_up()` loads `bootstrap_votes` from the persistent DB (including any `NotarizeVote` cast before the crash), then immediately broadcasts `SkipVote` for all slots in the current `first_nonannounced_window` range using only a `!voted_final` guard. Peers who received the pre-crash `NotarizeVote` now also receive a `SkipVote` from the same validator for the same slot.

📄 `validator/consensus/simplex/consensus.cpp`

```
// consensus.cpp ~line 82
void start_up() {
  ...
  if (auto window = bus->first_nonannounced_window) {
    for (size_t i = *window; i < last_slot; ++i) {
      auto slot = bus->slots.get(i);
      if (slot.has_value() && !slot->state->voted_final) {   // ← MUST also check !voted_notar
        owning_bus().publish<BroadcastVote>(SkipVote{i});
      }
    }
  }
}
```

Database persistence: The `test-consensus` harness uses `TestDbImpl` backed by `shared_ptr<DbInner>` (test-consensus.cpp:610, 689). The DB is preserved across gremlin kills, so `bootstrap_votes` correctly reflects pre-crash state on restart — matching production behaviour.

✓ **DYNAMICALLY CONFIRMED**

Gremlin parameters: `--gremlin-kills-leader`, period 2–4s, downtime 1–2s.

20 leader restarts observed, 19 showing immediate post-restart SkipVote equivocations.

Example: `Validator 0 restarted at 15:55:24.007` → `SkipVote{slot=5}` at `15:55:24.060` (53ms later) → `pool.cpp:525: slot already finalised`.

Recommended Fix

LOCATION	FILE : LINE	REQUIRED CHANGE
`alarm()`	consensus.cpp ~145	Add `&& !voted_notar && !voted_skip` to the guard before broadcasting `SkipVote`
`start_up()`	consensus.cpp ~87	Add `&& !voted_notar` to the guard before broadcasting `SkipVote`
`try_notarize()`	consensus.cpp ~232	Check `voted_skip` after `co_await ValidationRequest`; return early if set
`check_invariants()`	pool.cpp ~209	Add `notarize_ + skip_` case to misbehavior detection matrix

---

Reproduction

Docker (Recommended — no build required)

```
docker run --rm ghcr.io/tesseraeventures/ton-consensus-audit:latest
```

Runs in 30 seconds. Reproduces both alarm race and startup equivocation findings.

Source + Dockerfile: github.com/TesseraeVentures/ton-consensus-audit

Manual Build (Alternative)

PoC Script: `poc/test_equivocation_combined.py`
Repository: `github.com/TesseraeVentures/ton-consensus-audit`

Build

```
git clone https://github.com/ton-blockchain/ton
cd ton && git checkout testnet          # commit 3bb6abc
mkdir build-clang && cd build-clang
CC=clang CXX=clang++ cmake .. -DCMAKE_BUILD_TYPE=Release -DTON_USE_ROCKSDB=OFF
cmake --build . --target test-consensus -j$(nproc)
```

Run

```
# Both paths (recommended — ~90 seconds)
python3 poc/test_equivocation_combined.py --build-dir /path/to/ton/build-clang

# Path A only (~30 seconds)
python3 poc/test_equivocation_combined.py --build-dir /path/to/ton/build-clang --part 1

# Path B only (~60 seconds)
python3 poc/test_equivocation_combined.py --build-dir /path/to/ton/build-clang --part 2
```

Expected Output

```
PART 1: PASS — 11 equivocating (validator,slot) pairs  (8/8 validators affected)
PART 2: PASS — 19 post-restart equivocations           (20 gremlin restarts, 19 confirmed)

Overall: PASS
```

Affected Files

- `validator/consensus/simplex/consensus.cpp` — lines 82–91 (start_up), 144–150 (alarm), 228–235 (try_notarize)
- `validator/consensus/simplex/pool.cpp` — lines 209–217 (check_invariants), 525 (observable WARNING)